# NJOY REAL ESTATE DATABASE PROJECT
# -TECHNICAL REPORT-


| Team Members | Project Name |
| --- | --- |
| Altay Yeles<br>Alper Güner<br>Mustafa Ulaş Karaaslan<br>Mehmet Can Öztürk<br>Emre Aslan | Njoy Emlak — Real Estate Portfolio Management System<br>**Database Engine**<br>SQLite<br>**Data Source**<br>njoyemlak.com (Live Active Portfolio)<br>**Date**<br>April 2025 |


## 1. Introduction and Project Idea

This project was initiated to design a relational database for our team member Alper Güner's uncle's actively operating real estate company, Njoy Emlak (njoyemlak.com). The system focuses on consolidating listing, agent, and property data into a single normalized database and generating business intelligence reports from that data.

> *Data Collection Method: All listing data in the database was manually collected from njoyemlak.com (data scraping). 5 active listings along with all of their interior features, exterior features, and orientation details were entered into the system. Since the data comes from a real source, no artificial or fabricated content was used.*

## 2. Problem Statement and Importance

### 2.1 Problem Statement

Small and mid-sized real estate offices struggle to manage listing, agent, and feature data in an organized way as their portfolio grows. Dependency on scattered Excel files or ready-made platforms leads to key operational issues including data redundancy, update anomalies, and insufficient reporting capabilities.

### 2.2 Why This Project Matters

Implementing this relational database system delivers the following concrete benefits:

- Centralized Data Management: All listing, agent, and feature data is stored in one place; updates are made at a single point.

- Powerful Filtering: Instant queries by budget, area, district, and property features (elevator, AC, orientation, etc.) become possible.
- Performance Analysis: The number of listings per agent and the total portfolio value can be calculated instantly.
- Scalability: New listing types, feature categories, or staff members can be added to the system with ease.
- Platform Independence: SQLite file-based structure works in any environment and integrates with any application.

# 3. Database Schema Design

## 3.1 Tables and Relationships

Five tables have been designed targeting Third Normal Form (3NF). Relationships between tables are defined through Foreign Keys. The many-to-many (M:N) relationship between Listings and Features is resolved via a separate junction table.

| Table | Key Columns (PK / FK) | Description |
|---|---|---|
| **Ekip (Team)** | DanismanID (PK, AUTOINCREMENT) AdSoyad, Unvan, Telefon | Stores records of real estate agents and the store owner working at the office. |
| **Emlaklar (Listings)** | IlanID (PK) DanismanID (FK - Ekip) Baslik, Fiyat, Il, Ilce Mahalle, EmlakTipi BrutM2, NetM2, OdaSayisi | Holds the core details of all rental and sale property listings in the portfolio. |
| **Ozellik_Kategorileri (Feature Categories)** | KategoriID (PK, AUTOINCREMENT) KategoriAdi | Groups property features into categories: Interior, Exterior, Orientation. |
| **Ozellikler (Features)** | OzellikID (PK, AUTOINCREMENT) KategoriID (FK) OzellikAdi | A pool of all features such as Elevator, Air Conditioning, Steel Door, South-Facing. (33 features) |
| **Emlak_Ozellikleri (Listing-Features)** | IlanID (FK - Emlaklar) OzellikID (FK - Ozellikler) [Composite PK] | Junction table resolving the many-to-many (M:N) relationship between listings and features. |

## 3.2 Relationship Diagram

```
Ekip (1)  ————————< (N)  Emlaklar
                              |
                             (N)
                              v
                   Emlak_Ozellikleri   <- junction table
                              |
                             (N)
                              v
              Ozellikler (N) >——— (1) Ozellik_Kategorileri
```

*NOTE: The detailed version can be found in the SCHEMA.md folder.*

## 4. Planned SQL Features

The following SQL features are planned for implementation in upcoming phases of the project. Features marked "[planned]" have not yet been added to the current schema but are part of the architectural design.

| Planned SQL Feature | Purpose |
|---|---|
| **JOIN (INNER, LEFT, Multi-table)** | Joining tables; matching agents with their listings |
| **WHERE + IN / BETWEEN** | Multi-criteria filters by district, price range, and type |
| **GROUP BY + HAVING** | Per-agent portfolio analysis with filtered grouping |
| **COUNT, SUM, AVG** | Listing count, total portfolio value, average price per m² |
| **ORDER BY** | Sorting results by price, area, or date |
| **CTE [planned]** | Making complex queries modular, readable, and manageable with temporary result sets. |
| **VIEW [planned]** | Turning frequently used complex queries into reusable virtual tables |
| **INDEX [planned]** | Adding indexes on price and district columns for faster queries |

## 5. Sample SQL Queries

### Query 1 — Full Listing Report (INNER JOIN)

```
SELECT E.Baslik, E.Fiyat, E.Ilce, E.EmlakTipi,
       K.AdSoyad AS Danisman_Adi, K.Telefon
FROM Emlaklar E
INNER JOIN Ekip K ON E.DanismanID = K.DanismanID;
```

Lists all active listings with the responsible agent's name and contact information. This is the primary output query for clients or office managers.

### Query 2: Budget and Region Filtering (WHERE, IN)

```
SELECT Baslik, Fiyat, İlce, Mahalle
FROM Emlaklar
WHERE Fiyat <= 40000 AND İlce IN ('Şişli', 'Beyoğlu')
ORDER BY Fiyat DESC;
```

The goal is to retrieve apartments with a budget of 40,000 TL or less, specifically in the Şişli and Beyoğlu districts (İstiklal, Cihangir, Firuzağa, etc.).

### Query 3 — Agent Portfolio Analysis (GROUP BY + SUM + COUNT)

```
SELECT K.AdSoyad,
       COUNT(E.IlanID)  AS ToplamIlanSayisi,
       SUM(E.Fiyat)     AS ToplamPortfoyDegeri
FROM Ekip K
LEFT JOIN Emlaklar E ON K.DanismanID = E.DanismanID
GROUP BY K.AdSoyad;
```

A performance report designed for management. The LEFT JOIN ensures agents with no listings are also included; each agent's listing count and total portfolio value (TRY) is calculated automatically.

### Query 4 — Feature-Based Filtering (4-Table JOIN + DISTINCT)

```
SELECT DISTINCT E.Baslik, E.Fiyat, Oz.OzellikAdi, Kat.KategoriAdi
FROM Emlaklar E
INNER JOIN Emlak_Ozellikleri EO   ON E.IlanID     = EO.IlanID
INNER JOIN Ozellikler Oz          ON EO.OzellikID = Oz.OzellikID
INNER JOIN Ozellik_Kategorileri K ON Oz.KategoriID = K.KategoriID
WHERE Oz.OzellikAdi IN ('Asansor', 'Klima');
```

Lists listings with elevator or air conditioning along with their feature category. This query, combining four tables through chained JOINs, demonstrates the practical value of the junction table.

### Query 5 — Rent Cost per m² (Arithmetic)

```
SELECT Baslik, Fiyat, BrutM2, NetM2,
       ROUND(Fiyat / BrutM2, 0) AS BrutM2_Fiyati,
       ROUND(Fiyat / NetM2, 0)  AS NetM2_Fiyati
FROM   Emlaklar
WHERE  BrutM2 > 0 AND NetM2 > 0
ORDER BY NetM2_Fiyati DESC;
```

Calculates the monthly rent cost per gross and net square meter for each listing.

## 6. Conclusion

In this Milestone phase, the normalized database schema built with real data from the Njoy Emlak platform, along with table relationships and core queries, has been successfully completed. In upcoming phases, advanced SQL features such as CTE, VIEW and INDEX will be integrated into the system; the dataset will grow alongside the actual portfolio.

---

*Group's GitHub repository link: https://github.com/alpergnr/SQL-PROJECT*

---